

차량 OTA 부하에 대한 Kubernetes 확장·장애 대응

가상 차량 시뮬레이션 기반 단계적 배포와 서비스 인프라 구축

01 차량 OTA와 DevOps 과제

- 1-1. 차량 OTA의 개념과 필요성
- 1-2. Campaign 기반 단계적 Rollout
- 1-3. OTA 환경의 DevOps 과제

02 기술 설계 및 아키텍처

- 2-1. 초기 전체 아키텍처
- 2-2. 배포 & 관측 흐름

03 프로젝트 목표

- 3-1. 프로젝트 전제 조건과 한계
- 3-2. 목표 및 예상 결과

04 진행도와 협업

- 4-1. 프로젝트 전략

01 차량 OTA와 DevOps 과제

차량 OTA의 개념과 필요성

서비스 센터 방문 없이 차량 소프트웨어와 펌웨어를 배포하는 과정

CONCEPT

OTA

Over-The-Air

차량 SW·펌웨어 원격 업데이트

원격

무선

업데이트

OTA 분류

SOTA

Software OTA

인포테인먼트 · 내비게이션 · 앱

기능·편의 개선

FOTA

Firmware OTA

ECU · 배터리 · 제어 로직

성능·안전 개선

WHY OTA?

다수 차량 · 비동기 요청 · 단계적 Rollout



확장 · 장애 대응 · 관측

01 차량 OTA와 DevOps 과제

Campaign 기반 단계적 Rollout

캠페인 정책에 따라 대상 선정부터 결과 보고까지 수행

차량 OTA 기본 흐름

01 캠페인 생성

배포 버전·정책 등록

03 패키지 전달·검증

다운로드·checksum 확인

05 결과 보고

성공·실패와 상태 수집

02 대상·조건 판정

차량 정보와 설치 조건 확인

04 설치·재시작

안전 조건 충족 후 실행

CAMPAIGN

배포 정책

특정 차량에 어떤 버전을 어떤 조건과 순서로 배포할지 정의

대상 차종·연식·생산 공장·HW/FW 버전

일정 시작 시각·단계별 비율·배포 순서

설치 조건 시동 OFF·P단·배터리·네트워크

STAGED ROLLOUT

단계 + 관찰 + 승격

소규모 차량군의 결과를 확인한 뒤 다음 단계로 확대



자동 중단 성공률·오류율·설치 실패 지표가 임계치를 넘으면 다음 단계 승격 중단

01 차량 OTA와 DevOps 과제

OTA 환경의 DevOps 과제

Description

왜 어려운가		대응방향
비동기 요청	차량이 언제, 얼마나 몰릴지 예측 불가	Stateless 서버, 상태는 DB
트래픽 증가	단계 전환마다 요청이 계단식으로 증가	HPA, 예비 노드, 캐시
부분 실패와 재시도	일부만 실패, 동시 재시도가 또 폭주	상태 기록, 백오프 재시도
노드, 저장소 장애	노드나 NFS가 죽어도 서비스 유지	재스케줄링, 백업·복구
장애 원인 분석	서버·Pod가 많아 문제 위치 찾기 어려움	Metric · Log · Alert

단계적 Rollout이 다루는 DevOps 과제

- 장애 범위 제한

결함이 있어도 일부 차량군만 영향
- 실환경 검증

소규모 그룹에서 실제 실패 원인 확인
- 자동 중단

관측 지표가 임계치를 넘으면 승격 차단
- 인프라 부하 분산

Cache와 Origin의 순간 요청 집중 완화

02 기술 설계 및 아키텍처

초기 전체 아키텍처

Description

ORBIT

3개의 호스트 서버 + 외부 시뮬레이터

ASTRA Kubernetes · CI/CD · Monitoring

SOL OTA Package Cache

TERRA Campaign DB · Artifact Storage

LUNA Vehicle Simulators

실험 경계 VM 기반 독립 노드 모사 · 물리 HA 제외

GitHub

배포 설정 및 레지스트리 관리

Application Repository

Container Registry

GitOps Repository

ORBIT-ASTRA

Service Cluster 및 외부 구성

192.168.200.200

kubeadm Kubernetes Cluster

Control Plane VM
kubeadm

Cluster Workloads

Envoy · MetaLB

Argo CD · Alloy

Front-End

Back-End

Worker1 VM

Worker2 VM

Worker3 VM

CORDON · RESERVE



Jenkins VM



NFS VM



Vehicle DB VM



Monitoring VM

ORBIT-SOL

CDN Server

192.168.200.201



Edge Cache
Simulator VM

ORBIT-TERRA

Campaign 및 Origin File 관리

192.168.200.202



Campaign DB
VM



File Storage
VM

ORBIT-LUNA

차량 Container 운영

L2TP/IPsec



Vehicle
Simulators

02 기술 설계 및 아키텍처

배포 및 관측 흐름

GitOps 배포와 서비스-인프라 관측 경로

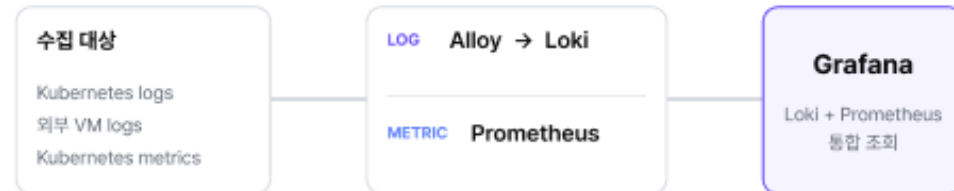
배포 흐름

main 반영부터 Pod 실행까지 GitOps로 자동화

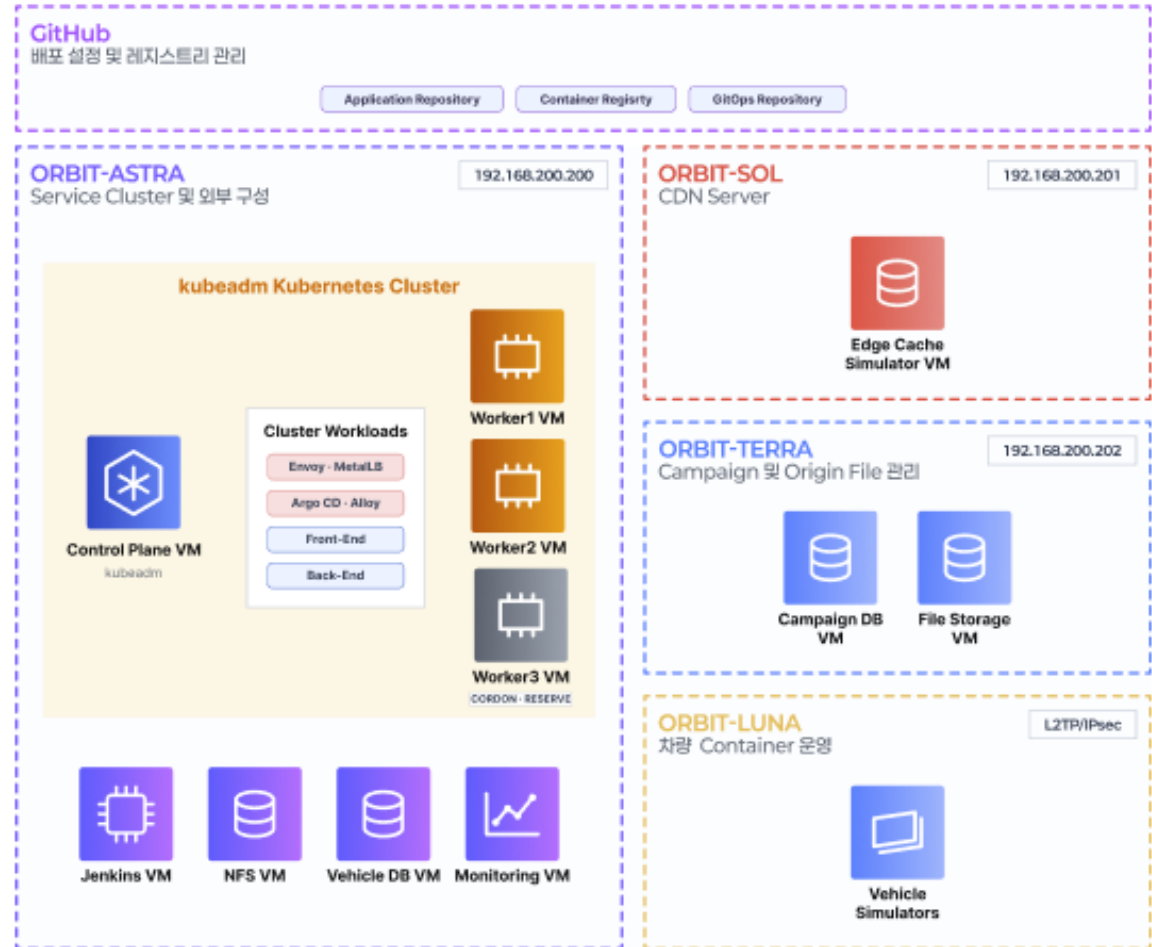


관측 흐름

로그와 메트릭을 분리 수집하고 Grafana에서 함께 조회



Kubernetes: Alloy DaemonSet · 외부 VM: Alloy systemd



03 프로젝트 목표

프로젝트 전제 조건과 한계

Description

01

가상 차량

Vehicle Simulator 기반의 임의 트래픽
발생 유도

02

상황 별 시나리오 설정

실제 OTA의 세부 과정은 공개되지 않음
현재 상황의 제약 조건에 맞는 인프라 구성 및 실험

03

모니터링 및 장애 대응

가상의 두 액터를 설정함
가상으로 OTA가 운영되며 시나리오 적용

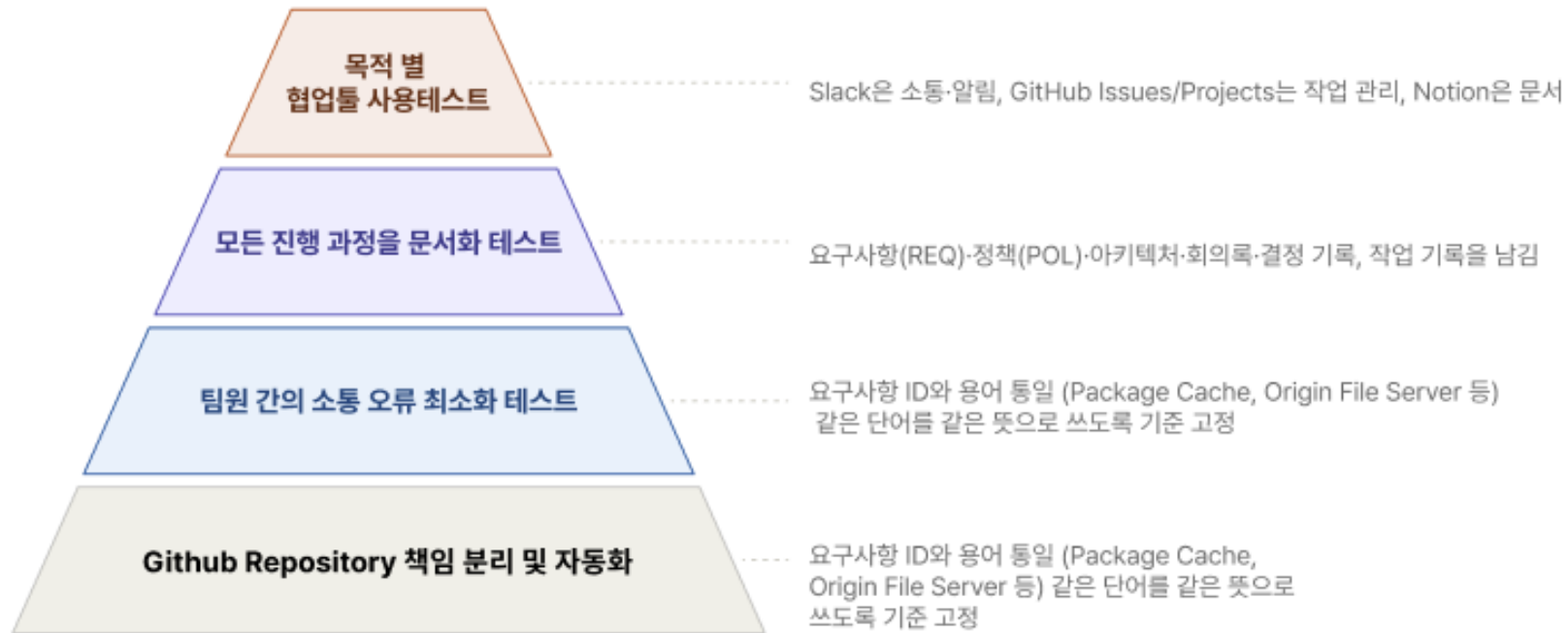
03 프로젝트 목표

목표 및 예상 결과

Description

	방법	측정 지표	기대효과
단계별 부하	차량 수를 단계적으로 증가	요청량, p95 응답시간	병목 지점 식별
pod 확장	HPA	scale-out 반응 시간	부하 증가에도 응답 시간 유지
노드 확장	Pending 발생 시 worker3 투입	투입 전후 응답시간	노드 부족 해소 확인
캐시 효과	SOL on/off로 비교	origin 요청 수	origin 부하 감소
자동 중단	결함 버전 배포	중단까지 시간, 영향 차량 수	피해를 초기 단계에서 차단
노드 장애	worker 1대 종료	Pod 재배포치 시간, 에러율	서비스 유지 확인
저장소장애	NFS 중단 후 복구	영향 범위, 복구 시간	단일 장애점 한계 수치화

04 진행도와 협업
프로젝트 전략
Description



04 진행도와 협업

프로젝트 전략

Description

